

Lab 6 - AI Practices with Codex

Student Sheet

Duration: 60 minutes

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Goal

The campus library lends laptops and charges a fee when one comes back late. A teammate's coding agent wrote `late_fee` in `fees.py` and reports that its tests pass. Before the change is accepted, you want two independent checks from two personas: a tester that writes tests from `REQUIREMENTS.md` and cannot see `fees.py`, and a reviewer that reads `fees.py` against `REQUIREMENTS.md` and changes nothing. You give Codex the project rules, define both personas, make Codex enforce their limits, and record a decision based on their results.

Setup

1. You need your MacBook with the ChatGPT desktop app, signed in, Python 3, Git and your editor, as in earlier labs. The app includes Codex, which you run in Terminal. Before the lab, the instructor will announce the exact Codex build tested for this sheet; permission profiles are beta and a later build is not automatically compatible.
2. Download and extract `lab06_files.zip`, then copy everything inside `lab06_files` into `lab06/` in your private `ai1220` repository, including the hidden `.gitignore` (Cmd+Shift+. shows it in Finder), so that `ai1220/lab06/fees.py` exists.
3. Open Terminal in `ai1220/lab06`, where you run every command on this sheet. Commit the supplied files once with `git add .` and `git commit -m "Start lab06"`, so that `git restore fees.py`, for example, can bring back a changed file.
4. Run this line on its own, to point `codex` at the Codex inside the ChatGPT app:

```
alias codex=/Applications/ChatGPT.app/Contents/Resources/codex-cli/bin/codex
```

Run `codex --version`. It must match the supported build announced by the instructor; if it differs, stop and ask before continuing. Record the version for the evidence file of Exercise 3. Start Codex once with `codex`. When Codex asks *Trust this folder?*, answer yes: Codex uses the folder's `.codex/config.toml` only in a trusted folder. If it asks you to sign in, sign in with your ChatGPT account. Leave with `/quit`. The alias lasts for one Terminal window: in a new window or tab, run the `alias` line again.

Keep `REQUIREMENTS.md` and `fees.py` as supplied: the lab tests the teammate's code. AI assistance is allowed, but check and understand your work.

Key concepts

Codex	OpenAI's coding agent, included in the ChatGPT app. Started with <code>codex</code> in a folder, it takes a request, runs commands on your Mac inside a sandbox, edits files and repeats until the task is done.
<code>AGENTS.md</code>	Rules for every agent in the folder. Codex adds the <code>AGENTS.md</code> files from the repository root down to the folder it runs in to each agent's instructions.
Custom agent	<code>.codex/agents/NAME.toml</code> with <code>name</code> , <code>description</code> and <code>developer_instructions</code> . When a request asks for the agent by name, Codex starts it as a subagent with a thread of its own, and it follows its developer instructions. A persona in this lab is a custom agent that runs in a session started with its permission profile.

Permission profile	A named set of limits in <code>.codex/config.toml</code> . The first line of that file, <code>default_permissions = "NAME"</code> , sets the profile Codex applies to everything it runs in the folder, including the agents it starts; <code>codex -c 'default_permissions="OTHER"'</code> starts a session with another profile. An agent gets the rules of <code>AGENTS.md</code> only when the profile reads <code>AGENTS.md</code> . Each rule gives "read" or "write" (read and change) to a path in the folder, such as "tests"; a path that no rule names can be neither read nor changed. <code>":minimal" = "read"</code> lets commands use the tools of macOS.
Network	Commands under a profile cannot reach the network unless the profile allows it. <code>web_search = "disabled"</code> turns off Codex's own web search.
Sandbox check	<code>codex sandbox -- COMMAND</code> , run in the folder, runs one command under the folder's default profile, without a model; <code>codex sandbox -P NAME -- COMMAND</code> runs it under the profile NAME.

Exercise 1: Project rules and two personas

35%

Create `AGENTS.md` in `lab06` with the rules for every agent: `REQUIREMENTS.md` holds the requirements; tests use `unittest`, live in `tests/` and import the function with `from fees import late_fee`; they run from `lab06` with `python3 -m unittest discover -s tests`; only the standard library is used.

Then create two custom agents in `.codex/agents/` and give each its role, its input, its output and when it stops. Create the folder in Terminal with `mkdir -p .codex/agents` (Finder rejects names that start with a dot) and write the files in your editor.

- `tester.toml`, the agent `tester`: input `REQUIREMENTS.md`; output `tests/test_fees.py`, with one test for each example in `REQUIREMENTS.md`, each checking the exact fee; it stops once the file exists, without running the tests.
- `reviewer.toml`, the agent `reviewer`: input `REQUIREMENTS.md` and `fees.py`; output a list, in its reply, of each place where `late_fee` breaks a criterion, with the criterion, an input, the fee the code returns and the fee required; it changes no file and stops after the list.

For example, an agent that explains a file:

```
# .codex/agents/explainer.toml
name = "explainer"
description = "Explains a file in plain words for a new teammate"
developer_instructions = """
You are the explainer. Read the file named in the request and explain what it does
in at most five sentences. Change no file. Stop after the explanation.
"""
```

Exercise 2: Limits that Codex enforces

25%

Create `.codex/config.toml` from this outline and write the rules of each profile under its `":workspace_roots"`, one per line, so that Codex itself enforces these limits:

- `tester`: reads `AGENTS.md`, `REQUIREMENTS.md` and the files in `tests/`; creates and changes files in `tests/` only; no other file in `lab06`.
- `reviewer`: reads `AGENTS.md`, `REQUIREMENTS.md` and `fees.py`; changes no file; no other file in `lab06`, so its review does not depend on the tests.

```

default_permissions = "tester"
web_search = "disabled"

[permissions.tester]
description = "The tester: reads the requirements, writes only tests/, no network"

[permissions.tester.filesystem]
":minimal" = "read"

[permissions.tester.filesystem.":workspace_roots"]
# the tester's rules: "PATH" = "read" or "PATH" = "write"

[permissionsReviewer]
description = "The reviewer: reads the code and the requirements, changes nothing"

[permissionsReviewer.filesystem]
":minimal" = "read"

[permissionsReviewer.filesystem.":workspace_roots"]
# the reviewer's rules

```

Before Exercise 3, run the commands of *How to check your work*.

Exercise 3: Two independent checks and the decision

40%

In each Codex session, keep the permissions Codex starts with and decline every request to run a command outside the sandbox.

1. Quit any open Codex session, start `codex` in `lab06` and ask it to spawn the tester subagent to write `tests/test_fees.py`: Codex starts a custom agent only when a request asks for it by name. State the goal, the context, the constraints and when the task is done, and keep a copy of your request for `DECISION.md`. The tester works in a thread of its own; when Codex reports that it has finished and `tests/test_fees.py` exists, leave with `/quit`. If the file is missing, ask again; after two tries, write the tests yourself and go on.
2. Compare each test with `REQUIREMENTS.md`: the input, the exact expected fee, and only inputs the requirements allow (whole numbers of minutes, 0 or more). Correct wrong tests and add any missing example yourself.
3. Run the tests with `python3 -m unittest discover -s tests`. Tests that fail on the teammate's `fees.py` are the evidence you need.
4. Commit the tests and note the short hash that the last command prints:

```

git add tests
git commit -m "Add the tester's tests for late_fee"
git log -1 --format=%h

```

5. Start a session with the reviewer's profile and ask Codex to spawn the reviewer subagent to review `fees.py` against `REQUIREMENTS.md`. Compare its findings with your failing tests, then leave with `/quit`.

```

codex -c 'default_permissions="reviewer"'

```

6. After running *How to check your work*, create `evidence/sandbox-checks.txt`. Write PASS only for a check whose observed result matches the sheet; otherwise write FAIL and the result you saw. Use these lines:

```
Codex version: output of codex --version
Tester reads AGENTS.md and REQUIREMENTS.md: PASS or FAIL
Tester writes tests/check.txt: PASS or FAIL
Tester cannot read fees.py or README.md: PASS or FAIL
Tester cannot change fees.py: PASS or FAIL
Tester cannot reach example.com: PASS or FAIL
Reviewer reads AGENTS.md and fees.py: PASS or FAIL
Reviewer cannot read tests/check.txt: PASS or FAIL
Reviewer cannot change fees.py: PASS or FAIL
```

7. Write `DECISION.md` with these seven lines, each completed from your own work:

```
Decision: accept, or request changes
Failing criteria: the criteria whose tests fail, for example AC1, AC3, or none
Evidence: one failing call, as late_fee(N), what it returned and what was expected
Review: the reviewer's finding on the same criterion, in one sentence
Tests commit: the short hash from step 4
Tester request: the request you gave Codex for the tester in step 1
Codex version: the output of codex --version
```

How to check your work

Run these commands in `lab06`, in a Terminal window where you ran the `alias` line of Setup, step 4. They try both profiles without a model. Each command of this block must print the file or `x`:

```
codex sandbox -- cat AGENTS.md
codex sandbox -- cat REQUIREMENTS.md
codex sandbox -- sh -c 'echo x > tests/check.txt && cat tests/check.txt'
codex sandbox -P reviewer -- cat AGENTS.md
codex sandbox -P reviewer -- cat fees.py
```

Each command of this block must fail with *Operation not permitted*, *No such file or directory* or a network error:

```
codex sandbox -- cat fees.py
codex sandbox -- cat README.md
codex sandbox -- sh -c 'echo x >> fees.py'
codex sandbox -- curl -sS https://example.com
codex sandbox -P reviewer -- cat tests/check.txt
codex sandbox -P reviewer -- sh -c 'echo x >> fees.py'
```

Then run `rm tests/check.txt`, and `git restore fees.py` if a command changed `fees.py`. Record the observed results in `evidence/sandbox-checks.txt`; do not record PASS merely from the expected result printed here.

Deliverables

Before the lab ends, commit and push `lab06`, then check GitHub. Required work: `AGENTS.md`, both files in `.codex/agents/`, `.codex/config.toml`, `tests/test_fees.py`, `evidence/sandbox-checks.txt` and `DECISION.md`. Keep `REQUIREMENTS.md` and `fees.py` unchanged; remove `tests/check.txt` and do not add Codex session files.

```
cd ..
git add lab06
git commit -m "Complete lab06 late fee checks"
git push
git rev-parse HEAD
```

Before the lab ends, submit the private GitHub repository URL and full commit hash through the LMS. Ensure the instructor can open it; do not upload `lab06_files.zip`.

References: [OpenAI Codex configuration](#); [permissions](#); [Python unittest](#).